

Deep Q-Learning

Development Journal

Project Topic: Implementation and Study of a Deep Q-Network (DQN) with Double and Dueling Extensions

My project topic is the implementation and study of Deep Q-Learning, a reinforcement learning algorithm that trains a neural network to play games through trial and error. The project trains an agent on two Gymnasium environments — CartPole and Flappy Bird — and this journal focuses on the technical details of the algorithm, including the Bellman equation, epsilon-greedy exploration, experience replay, target networks, and the Double DQN and Dueling DQN extensions.

Acknowledgement: The implementation in this repository was built by following a YouTube tutorial series on Deep Q-Learning with PyTorch, and was then adapted and restructured. This study and report were undertaken with the assistance of AI, which was used to break down the theoretical components of the model (e.g., the Bellman equation, epsilon decay, target networks) and to relate them back to the code. All explanations were reviewed against the original research papers and the working implementation to ensure correctness.

Table of Contents

1. Motivation	3
2. How Did I Study?	3
3. Definition	3
3.1 Reinforcement Learning	4
3.2 Q-Learning	4
3.3 Deep Q-Network (DQN)	4
4. Hypothesis / Prediction Function	5
4.1 The Q-Function	5
4.2 The Prediction Function	5
4.3 Hypothesis Space	6
5. Key Components of the Model	6
5.1 Inputs	6
5.2 Output	6

5.3 Parameters	7
5.4 The Bellman Equation and the TD Target	7
5.5 Epsilon-Greedy Exploration	8
5.6 Experience Replay	9
5.7 The Target Network	9
5.8 Double DQN	9
5.9 Dueling DQN	10
5.10 Loss Function and Optimisation	10
6. Training & Prediction Processes	11
6.1 Training Process	11
6.2 Prediction Process	12
7. Critical Analysis and Discussion of DQN	12
7.1 Advantages	12
7.2 Limitations	12
7.3 Challenges	13
7.4 Other Considerations	13
8. Implementation	14
8.1 Repository Structure and Libraries	14
8.2 The Q-Network (dqn.py)	14
8.3 Experience Replay (experience_replay.py)	15
8.4 The Agent and Training Loop (agent.py)	15
8.5 Action Selection	16
8.6 The Optimisation Step	16
8.7 Hyperparameter Configuration	16
9. Discussion — Expected Training Behaviour	17
10. Conclusion & Reflection	18
11. Appendix A — Glossary	18
12. Appendix B — References	19

1. Motivation

I chose to study Deep Q-Learning because reinforcement learning is fundamentally different from the supervised learning I have worked with before. In my previous project (an ID3 decision tree) the model learned from a labelled dataset where every sample had a correct answer. In reinforcement learning there is no labelled data at all: the agent must discover good behaviour by interacting with an environment and observing the rewards its actions produce. Understanding how a model can learn from delayed, indirect feedback felt like an important next step in my machine learning education.

DQN is also historically significant. It was the algorithm that launched the modern field of deep reinforcement learning when DeepMind used it to reach human-level performance on Atari 2600 games from raw inputs (Mnih et al., 2015), and its ideas — experience replay, target networks, and their later refinements — still underpin value-based methods today. Flappy Bird makes an appealing testbed because it is easy to visualise, has only two actions (flap or do nothing), and still demands precise, well-timed decisions.

Finally, I wanted to properly understand the parts of the code I had copied but found mysterious: what the epsilon values actually control, why the agent keeps a memory of past experiences, and why there are two copies of the same neural network. This journal is my attempt to close that gap.

2. How Did I Study?

This project began differently from my ID3 project: rather than building the algorithm from scratch first, I started from a working implementation that I copied from a YouTube tutorial series on Deep Q-Learning with PyTorch. The tutorial built the agent incrementally against Gymnasium environments, which gave me a running system, but it also meant I initially had code I could not fully explain. My study process was therefore about working backwards from working code to the theory underneath it.

I used AI as a learning aid to decompose the algorithm into its parts — the Q-function, the Bellman equation, epsilon-greedy exploration, experience replay, the target network, and the Double and Dueling extensions — and asked it to connect each concept to the specific lines in `agent.py` and `dqn.py` that implement it. I then checked these explanations against the original research papers (Mnih et al., 2015; van Hasselt et al., 2016; Wang et al., 2016) and Sutton and Barto's textbook to confirm they were accurate.

I have to be honest that I have not yet run the long training sessions that Flappy Bird requires — only short exploratory runs. For that reason this journal replaces a results section with a discussion of what the training process does and what I expect to happen when it is run properly (Section 9). The cycle of reading code, questioning it, and tracing tensors through the network built both my confidence and my understanding of the algorithm.

3. Definition

A Deep Q-Network (DQN) is a reinforcement learning model that uses a neural network to estimate how good each possible action is in a given situation. The result is an agent that, at every moment, scores its available actions and can act by picking the highest-scoring one.

3.1 Reinforcement Learning

- Reinforcement learning (RL) is a framework where an **agent** learns by interacting with an **environment** over a sequence of time steps.
- At each step the loop is:
 - the agent observes the current **state** of the environment,
 - chooses an **action**,
 - receives a **reward** (a number indicating how good that step was),
 - and the environment transitions to a new state.
- An **episode** is one complete play-through, from reset until a terminal state (e.g., the bird crashing into a pipe).
- The goal is to maximise the total **discounted** reward collected over time — not just the immediate reward, but future rewards too.
- Unlike supervised learning there are no labelled examples; the reward signal is the only feedback, and it may arrive long after the action that caused it.

3.2 Q-Learning

- Q-Learning (Watkins, 1989) is a **value-based** RL method: instead of learning a policy directly, it learns a value function $Q(s, a)$ that estimates the total future reward of taking action a in state s and acting optimally afterwards.
- Classic (tabular) Q-learning stores one Q-value per state–action pair in a table and updates it after every step using the Bellman equation (Section 5.4).
- Once the Q-values are accurate, the best policy is simple: in every state, pick the action with the highest Q-value.
- The tabular approach only works when the number of states is small and discrete. Flappy Bird's state is 12 continuous numbers, so a table is impossible — there are effectively infinitely many states.

3.3 Deep Q-Network (DQN)

- DQN (Mnih et al., 2015) replaces the Q-table with a neural network $Q(s, a; \theta)$ whose weights θ are learned. The network takes the state vector as input and outputs one Q-value per action.
- Because the network generalises, it can produce sensible Q-values for states it has never seen — nearby states produce similar outputs.
- Training a neural network on its own predictions is unstable, so DQN introduced two stabilising mechanisms:
 - **experience replay** — learning from a memory of stored past transitions (Section 5.6),
 - a **target network** — a frozen copy of the network used to compute learning targets (Section 5.7).
- This project also implements two later refinements, toggled per run in the configuration file:
 - **Double DQN** (van Hasselt et al., 2016) — reduces overestimation of Q-values (Section 5.8),

- **Dueling DQN** (Wang et al., 2016) — splits the network into state-value and advantage streams (Section 5.9).

4. Hypothesis / Prediction Function

4.1 The Q-Function

The quantity the model tries to learn is the **return**: the total reward from time step t onwards, where future rewards are shrunk by a discount factor γ (gamma) between 0 and 1:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Where:

- G_t is the return — the discounted sum of all future rewards
- r_{t+1} is the reward received after acting at time step t
- γ (gamma) is the discount factor; $\gamma = 0.99$ in this project, meaning a reward k steps in the future is worth 0.99^k of an immediate one — the agent cares about the future but prefers rewards sooner

The Q-function is the expected return for taking action a in state s and then following the policy π afterwards:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$$

Where:

- $Q^\pi(s, a)$ is the action-value: "how much total reward do I expect if I do a now, and act according to π from then on?"
- $\mathbb{E}_\pi[\cdot]$ is the expectation (average over the randomness of the environment and the policy)
- s_t, a_t are the state and action at time step t

4.2 The Prediction Function

Where my ID3 decision tree mapped a feature vector directly to a class label, the DQN's hypothesis is a learned function $Q(s, a; \theta)$ — a neural network with weights θ that maps a state to a **vector of scores, one per action**. The actual prediction (which action to take) is obtained by taking the argmax over that vector:

$$h(s) = \arg \max_a Q(s, a; \theta)$$

Where:

- $h(s)$ is the action the trained agent takes in state s
- $Q(s, a; \theta)$ is the network's estimated value of action a in state s
- θ (theta) is the set of all network weights — these are the parameters learning adjusts

In Flappy Bird the action set is {0 = do nothing, 1 = flap}, so the network outputs two numbers and the agent flaps whenever the flap Q-value is the larger of the two. During training this greedy rule is deliberately softened by epsilon-greedy exploration (Section 5.5).

4.3 Hypothesis Space

The hypothesis space is the set of all functions the model could possibly represent. For ID3 this was the (finite but enormous) set of decision trees over the discretised features. For a DQN it is the set of all weight assignments θ of the chosen architecture — a **continuous** space, so there are uncountably many hypotheses.

- The architecture determines the expressiveness of the space: the shared hidden layer width (`fc1_nodes` = 512 for Flappy Bird) and the dueling stream width (`dueling_hidden_dim` = 256) set how complex a Q-function the network can represent.
- Where ID3 searched its space greedily and discretely (one split at a time), DQN searches its space by **gradient descent** — smoothly nudging θ in the direction that reduces prediction error.
- A consequence is that DQN training is stochastic and path-dependent: different random seeds explore different trajectories through weight space and can land on different final policies, unlike ID3 which always produces the same tree from the same data.

5. Key Components of the Model

5.1 Inputs

The input to the network is the environment's **observation**: a fixed-length vector of numbers describing the current state. No manual feature engineering or discretisation is needed (unlike ID3, which required binning continuous values) — the network consumes the raw floats directly.

- **FlappyBird-v0** (with `use_lidar: False`) provides 12 features:
 - the last pipe's horizontal position, and its top and bottom vertical positions,
 - the next pipe's horizontal position, and its top and bottom vertical positions,
 - the next-next pipe's horizontal position, and its top and bottom vertical positions,
 - the player's vertical position, vertical velocity, and rotation.
- **CartPole-v1** provides 4 features: cart position, cart velocity, pole angle, and pole angular velocity.
- In the code the sizes are read from the environment rather than hardcoded:
`env.observation_space.shape[0]` and `env.action_space.n`.

5.2 Output

The output layer has one node per action, and each node's value is the estimated Q-value of that action in the input state. For both environments used here that means two outputs.

- The outputs are **not probabilities** — they are estimates of total discounted future reward, and can be any real number (early Flappy Bird Q-values are negative, because random play mostly ends in the -1 death penalty).

- A single forward pass evaluates every action at once; acting greedily is just an **argmax** over the output vector.

5.3 Parameters

It is worth separating two kinds of "parameters". The **learned parameters** are the network weights θ , adjusted automatically by training. The **hyperparameters** are chosen beforehand and control how learning proceeds; in this project they live in `hyperparameters.yml`, with one named set per run:

Hyperparameter	Symbol	cartpole1	flappybird1	Role
learning_rate_a	α	0.001	0.0001	Step size of each gradient update
discount_factor_g	γ	0.99	0.99	How much future rewards are worth
epsilon_init	ϵ_0	1.0	1.0	Initial exploration rate (100% random)
epsilon_decay	d	0.9995	0.99995	Multiplicative decay applied to ϵ
epsilon_min	$\epsilon_{\min}$	0.01	0.05	Exploration floor — never fully greedy
replay_memory_size	—	100,000	100,000	Max transitions kept in replay memory
mini_batch_size	N	64	32	Transitions sampled per learning step
network_sync_rate	—	100	10	Env steps between target-network syncs
fc1_nodes	—	128	512	Width of the shared hidden layer
dueling_hidden_dim	—	128	256	Width of value/advantage streams
enable_double_dqn	—	False	True	Double DQN target on/off
enable_dueling_dqn	—	True	True	Dueling architecture on/off
stop_on_reward	—	100,000	100,000	Per-episode reward cap (safety valve)
seed	—	42	42	RNG seed for reproducibility

The two environments are tuned differently: Flappy Bird uses a smaller learning rate and a much slower epsilon decay because it is a harder, sparser-reward task that needs far more exploration, plus a wider network and the Double DQN correction.

5.4 The Bellman Equation and the TD Target

The mathematical heart of Q-learning is the **Bellman optimality equation**. It says that the value of acting optimally now equals the immediate reward plus the discounted value of acting optimally from wherever you land:

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]$$

Where:

- $Q^*(s, a)$ is the optimal Q-value of taking action a in state s
- r is the immediate reward, and s' is the next state
- $\max_{a'} Q^*(s', a')$ is the value of the best action available in the next state

This recursive definition gives us a learning rule: whatever our current estimate says, the right-hand side computed from real experience is a slightly better estimate. Tabular Q-learning nudges its estimate toward it:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Where:

- α (alpha) is the learning rate — how far each update moves the estimate
- the bracketed quantity is the **temporal-difference (TD) error**: the gap between what we predicted and what the experience suggests

DQN uses exactly the same idea, but expressed as a regression target for the neural network. For each stored transition (s, a, s', r, d) it builds the **TD target**:

$$y = r + (1 - d) \gamma \max_{a'} \hat{Q}(s', a'; \theta^-)$$

Where:

- y is the target value the network is trained to output for (s, a)
- d is the termination flag (1 if the episode ended at this transition); the $(1 - d)$ factor zeroes out future value when the bird has crashed — there is no future after a terminal state
- $Q(\cdot; \theta^-)$ is the **target network** — a frozen copy of the weights (Section 5.7)

This is **bootstrapping**: the network learns from targets partly built out of its own (frozen) predictions. It is what lets the reward signal from a crash propagate backwards through time to the actions that caused it, one update at a time.

5.5 Epsilon-Greedy Exploration

If the agent always exploited its current Q -estimates, it would never discover actions whose value it currently underrates — an agent that has never passed a pipe has no idea that pipes are worth passing. If it always acted randomly, it would never refine its behaviour. Epsilon-greedy is the standard compromise: with probability ϵ act randomly (**explore**), otherwise act greedily (**exploit**):

$$a_t = \begin{cases} \text{random action from } \mathcal{A}, & \text{with probability } \epsilon \\ \arg \max_a Q(s_t, a; \theta), & \text{with probability } 1 - \epsilon \end{cases}$$

Where:

- ϵ (epsilon) is the exploration rate, between 0 and 1
- $\mathcal{A}$ is the action set (e.g., {do nothing, flap})

The key is that ϵ is not fixed — it follows a **decay schedule**. Training starts at $\epsilon = 1.0$ (pure random play, gathering diverse experience) and after every optimisation step ϵ is multiplied by a decay factor d , but never allowed below a floor:

$$\epsilon \leftarrow \max(\epsilon \cdot d, \epsilon_{min})$$

Where:

- d is `epsilon_decay` (0.99995 for Flappy Bird — very slow, because the game needs a long exploration phase)
- $\epsilon_{\min}$ is `epsilon_min` (0.05 for Flappy Bird): the agent keeps taking a random action 5% of the time forever, so it never completely stops exploring

With $d = 0.99995$, reaching the 0.05 floor takes roughly 60,000 optimisation steps ($0.99995^n = 0.05 \Rightarrow n \approx 59,900$) — this is why the epsilon curve in the training plot is a long smooth exponential slide rather than a quick drop. These were the "epsilon values" I originally found mysterious: ϵ is simply the probability of acting randomly, annealed from 1.0 down to 0.05 over the course of training.

5.6 Experience Replay

Each interaction with the environment produces a **transition** tuple (s, a, s', r, d): state, action, next state, reward, and whether the episode terminated. Instead of learning from each transition once and discarding it, DQN stores transitions in a large ring buffer — the **replay memory** — and learns from randomly sampled mini-batches of old and new experience (Lin, 1992; Mnih et al., 2015).

- In this project the memory is a `deque` with `maxlen=100,000`; once full, the oldest transitions fall out as new ones arrive.
- Random sampling matters for two reasons:
 - **decorrelation** — consecutive game frames are nearly identical, and training a network on a stream of highly correlated samples violates the i.i.d. assumption of stochastic gradient descent and destabilises learning; shuffling breaks the correlation,
 - **sample efficiency** — each transition can contribute to many gradient updates instead of one, which matters when experience is expensive to collect.
- Replay is what makes DQN an **off-policy** method: it learns about the greedy policy from data generated by an older, more random policy.

5.7 The Target Network

The TD target y contains a Q-value prediction. If the same network that is being trained also produced its own targets, every weight update would shift the targets too — the network would be chasing a moving goalpost, which in practice can oscillate or diverge.

- DQN's fix is a second network, the **target network** $Q(\cdot; \theta^-)$, which is an exact copy of the policy network's architecture whose weights are **frozen** between periodic syncs.
- Every `network_sync_rate` environment steps (10 for Flappy Bird), the policy network's weights are copied into the target network:
`target_dqn.load_state_dict(policy_dqn.state_dict())`.
- Between syncs the targets stay fixed, so each learning step is ordinary regression toward a stationary target — dramatically more stable.
- This answered my "why are there two networks?" question: only the policy network is trained; the target network is a periodically-refreshed snapshot used solely to compute targets.

5.8 Double DQN

The max operator in the standard TD target causes a systematic **overestimation bias**: Q-estimates are noisy, and taking the maximum over noisy estimates tends to pick values whose noise happens to be positive. The bias compounds through bootstrapping and can slow or destroy learning. Double DQN (van Hasselt et al., 2016) decouples action **selection** from action **evaluation**:

$$y^{DoubleDQN} = r + (1 - d) \gamma \hat{Q}(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-)$$

Where:

- the **policy network** (θ) chooses which next action looks best: $\arg \max_{a'} Q(s', a'; \theta)$
- the **target network** (θ^-) supplies the value of that chosen action

Because the two networks make independent errors, an action must look good to both to receive a high target — random upward noise in one network no longer inflates the target on its own. In the code this is the `enable_double_dqn` branch inside `optimize()`; it is enabled for Flappy Bird and disabled for CartPole, which is simple enough not to need it.

5.9 Dueling DQN

The dueling architecture (Wang et al., 2016) changes the network itself rather than the target. After the shared hidden layer, the network splits into two streams that are recombined into Q-values:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right)$$

Where:

- $V(s)$ is the **state value**: how good is this situation, regardless of what I do next?
- $A(s, a)$ is the **advantage**: how much better or worse is action a than the average action here?
- the subtracted mean (over the $|\mathcal{A}|$ actions) makes the decomposition identifiable — without it, any constant could be shifted between V and A and produce the same Q , leaving the two streams free to drift; forcing the advantages to average zero pins V down as the mean action value

The intuition: in most Flappy Bird frames the bird is gliding between pipes and the action barely matters — what matters is whether the state itself is good (aligned with the gap) or bad (about to collide). The V stream can learn that from every visit to the state, without needing every action tried there, while the A stream focuses capacity on the moments where the choice genuinely matters. In `dqn.py` this is the `fc_value/value` and `fc_advantages/advantages` pair of stream heads.

5.10 Loss Function and Optimisation

With targets in hand, learning is ordinary neural-network regression. The loss is the mean squared error between the network's predictions for the actions actually taken and their TD targets, over a mini-batch of N sampled transitions:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - Q(s_i, a_i; \theta))^2$$

Where:

- y_i is the TD target for the i -th sampled transition (computed under `torch.no_grad()` — gradients never flow through targets)
- $Q(s_i, a_i; \theta)$ is the policy network's current estimate for the action that was actually taken (selected with `gather`)
- N is `mini_batch_size` (32 for Flappy Bird)

Gradients of this loss are computed by backpropagation and applied by the **Adam** optimiser (Kingma & Ba, 2015) with learning rate α . The code notes that MSE could be swapped for the Huber (smooth L1) loss, which the original DQN paper effectively used via error clipping; Huber is less sensitive to the occasional huge TD error.

6. Training & Prediction Processes

6.1 Training Process

The training loop in `agent.py` ties all the components together. One full cycle looks like this:

1. **Initialise**
 - Build the policy network with random weights; copy it to create the target network; create an empty replay memory; set $\epsilon = 1.0$.
2. **Play an episode**
 - At each step, select an action ϵ -greedily, execute it in the environment, observe (s', r, d) , and append the transition (s, a, s', r, d) to the replay memory.
3. **Sample a mini-batch**
 - After the episode ends (and once the memory holds more than one batch worth of transitions), sample a random mini-batch from the replay memory.
4. **Compute targets**
 - For every sampled transition compute the TD target y using the target network — with the Double DQN rule if enabled.
5. **Update the policy network**
 - Compute the MSE loss between predicted Q-values and targets; backpropagate; take one Adam step.
6. **Decay and sync**
 - Multiply ϵ by the decay factor (respecting the floor); if enough environment steps have accumulated, copy the policy weights into the target network.
7. **Repeat indefinitely**
 - Training has no fixed end — it loops forever and is stopped manually. Whenever an episode sets a new best reward, the current weights are saved to `runs/<set>.pt`, so the checkpoint on disk is always the best policy seen so far.

One implementation detail worth noting: this code performs **one optimisation step per episode** (after the episode ends), whereas the original DQN performs one per environment step. This is a simplification that trades learning speed for simplicity — with long episodes it learns less per unit of experience collected.

6.2 Prediction Process

Once trained, using the model is much simpler — all of the training machinery (replay memory, target network, epsilon) disappears:

1. **Load the trained weights**
 - The saved checkpoint is loaded into the network and the network is put in evaluation mode (`policy_dqn.eval()`).
2. **Observe the state**
 - The current 12-feature observation is read from the environment.
3. **Forward pass**
 - The state is passed through the network, producing one Q-value per action.
4. **Act greedily**
 - The action with the highest Q-value is executed (ϵ is treated as 0 — no exploration at test time).
5. **Repeat**
 - Steps 2–4 loop until the episode ends. Running `python agent.py flappybird1` (without `--train`) does exactly this, with rendering turned on.

7. Critical Analysis and Discussion of DQN

7.1 Advantages

- Handles continuous, high-dimensional state spaces
 - The neural network generalises across similar states, so DQN works where tabular methods are impossible — no discretisation or binning of inputs is required.
- Learns without labelled data
 - The only supervision is the reward signal; the agent generates its own training data by playing. No human needs to demonstrate correct play.
- Sample reuse through experience replay
 - Each stored transition can contribute to many updates, extracting more learning from every interaction with the environment.
- Off-policy learning
 - DQN can learn about the greedy policy from data collected by an old, mostly random policy — exploration and learning are cleanly separated.
- Modular, well-understood extensions
 - Improvements like Double and Dueling DQN bolt on with small code changes (in this repo they are literally config toggles) and compose with each other.

7.2 Limitations

- Discrete actions only
 - The argmax over actions requires a finite action set. Continuous control (e.g., steering angles) needs different algorithms (DDPG, SAC).
- Sample hungry
 - Even simple games need tens of thousands of episodes. The epsilon schedule alone implies ~60,000 episodes before exploration reaches its floor on Flappy Bird.
- Training instability and hyperparameter sensitivity
 - With function approximation, bootstrapping, and off-policy learning combined (the "deadly triad", Sutton & Barto, 2018), convergence is not guaranteed; a poorly-chosen learning rate or sync rate can make learning collapse rather than merely slow down.
- Overestimation bias
 - The max operator systematically inflates value estimates; Double DQN reduces but does not fully eliminate this.
- Black-box policies
 - Unlike my ID3 tree, whose every prediction could be traced through human-readable rules, a DQN's decision is an opaque function of 100k+ weights — I can see *what* the agent does, but not read off *why*.

7.3 Challenges

- Tuning the exploration schedule
 - If ϵ decays too fast the agent commits to a bad policy before it has discovered better options; too slow and it wastes tens of thousands of episodes flapping randomly. The decay constant is the single hardest hyperparameter to get right in this project.
- Sparse and delayed rewards
 - Flappy Bird's environment rewards +1.0 for passing a pipe, +0.1 per frame survived, and -1.0 for dying. Early random play almost never passes a pipe, so the informative +1.0 signal is extremely rare, and credit must propagate backwards through many steps of bootstrapping.
- Silent failure modes
 - A supervised model with a bug usually shows an obviously wrong accuracy; a buggy RL agent often just... fails to improve, with no error message. Diagnosing whether the cause is a bug, a hyperparameter, or simply not enough training time is genuinely difficult.
- Compute and time
 - Long training runs are needed before you can even evaluate a change, which makes iteration slow — a practical constraint I ran into directly in this project.

7.4 Other Considerations

- Reproducibility
 - The implementation seeds Python, NumPy, PyTorch, and the environment RNGs from a config value, so a training run can be reproduced — though results remain sensitive to that seed choice.

- Loss function choice
 - MSE was used here for simplicity; the Huber loss is the standard robustness upgrade and is flagged as a possible swap in the code.
- Further extensions
 - This project implements two of the six components of the Rainbow agent (Hessel et al., 2018); the others — prioritised experience replay, multi-step returns, distributional RL, and noisy networks — are natural next steps.
- Educational value
 - DQN sits at the historical junction between classical Q-learning theory and modern deep RL, which makes it an ideal algorithm to study: every piece (Bellman targets, exploration, replay, target networks) reappears in more advanced methods.

8. Implementation

8.1 Repository Structure and Libraries

The implementation is split across three Python modules plus a configuration file:

- `agent.py` — the training/evaluation loop, epsilon-greedy action selection, and the optimisation step
- `dqn.py` — the Q-network (vanilla or dueling)
- `experience_replay.py` — the replay memory buffer
- `hyperparameters.yml` — one named hyperparameter set per run

The main libraries used:

```
import torch                # neural network, autograd, optimiser
from torch import nn        # layers and the MSE loss
import gymnasium as gym     # standard RL environment interface
import flappy_bird_gymnasium # registers the FlappyBird envs with gymnasium
import yaml                 # loads hyperparameters.yml
import numpy as np          # seeding + reward statistics
import matplotlib           # renders the training progress plots
```

8.2 The Q-Network (`dqn.py`)

The network is a small multi-layer perceptron. A shared fully-connected layer processes the state; if dueling is enabled the output splits into a value stream and an advantage stream which are recombined into Q-values:

```

def forward(self, x):
    x = F.relu(self.fc1(x))                # shared hidden layer

    if self.enable_dueling_dqn:
        v = F.relu(self.fc_value(x))      # value stream
        V = self.value(v)                  # -> scalar V(s)

        a = F.relu(self.fc_advantages(x)) # advantage stream
        A = self.advantages(a)             # -> one A(s,a) per action

        # Q(s,a) = V(s) + (A(s,a) - mean_a(A(s,a')))
        Q = V + A - torch.mean(A, dim=1, keepdim=True)
    else:
        Q = self.output(x)                 # plain DQN head

    return Q

```

This is a direct transcription of the dueling equation from Section 5.9. For Flappy Bird the layer sizes are 12 → 512 (shared) → 256 (each stream) → 1 value + 2 advantages.

8.3 Experience Replay (experience_replay.py)

The replay memory is deliberately minimal — a bounded double-ended queue plus uniform random sampling:

```

class ReplayMemory():
    def __init__(self, maxlen, seed=None):
        self.memory = deque([], maxlen=maxlen) # oldest items fall out
        if seed is not None:
            random.seed(seed)

    def append(self, transition):
        self.memory.append(transition)          # transition = (s, a, s', r, done)

    def sample(self, sample_size):
        return random.sample(self.memory, sample_size) # uniform, no replacement

```

Sampling is uniform: every stored transition is equally likely to be chosen. (Prioritised experience replay, which samples surprising transitions more often, would be the natural upgrade.)

8.4 The Agent and Training Loop (agent.py)

The `Agent` class reads its named hyperparameter set from the YAML file, then `run(is_training=True)` executes the loop from Section 6.1. Some details worth pointing out:

- Two networks are built: `policy_dqn` (trained) and `target_dqn` (frozen copy, synced every `network_sync_rate` steps).
- Transitions are stored as CPU tensors so a 100,000-item replay buffer never pins GPU memory; each sampled batch is moved to the device just-in-time.
- Episodes run inside `itertools.count()` — an infinite loop. Training is stopped manually, and the best-so-far model checkpoint is saved whenever an episode sets a new reward record.

- A progress graph (100-episode rolling mean reward, plus the epsilon curve) is refreshed to `runs/<set>.png` at most every 10 seconds.

8.5 Action Selection

Action selection implements the epsilon-greedy rule from Section 5.5 in four lines:

```
def _select_action(self, state, epsilon, policy_dqn, env, is_training):
    # Epsilon-greedy: explore with a random action, else exploit the policy
    if is_training and random.random() < epsilon:
        return env.action_space.sample()          # explore
    with torch.no_grad():                          # no gradients needed to act
        return policy_dqn(state.unsqueeze(dim=0)) \
            .squeeze().argmax().item()             # exploit: best Q-value
```

- `unsqueeze(dim=0)` adds a batch dimension (the network expects batches), and `squeeze()` removes it again from the output before the `argmax`.
- At evaluation time `epsilon` is passed as 0.0, so the random branch is never taken.

8.6 The Optimisation Step

The `optimize()` method converts a sampled mini-batch into one gradient update, implementing Sections 5.4, 5.8 and 5.10:

```
with torch.no_grad():                             # targets are constants
    if self.enable_double_dqn:
        # policy net SELECTS the action, target net EVALUATES it
        best_actions = policy_dqn(new_states).argmax(dim=1)
        target_q = rewards + (1 - terminations) * self.discount_factor_g * \
            target_dqn(new_states) \
                .gather(dim=1, index=best_actions.unsqueeze(dim=1)) \
                .squeeze()
    else:
        # standard DQN: max over actions from the target network
        target_q = rewards + (1 - terminations) * self.discount_factor_g * \
            target_dqn(new_states).max(dim=1)[0]

# Q-values of the actions actually taken, from the current policy
current_q = policy_dqn(states).gather(dim=1, index=actions.unsqueeze(dim=1)) \
    .squeeze()

loss = self.loss_fn(current_q, target_q)           # MSE
self.optimizer.zero_grad()                       # clear old gradients
loss.backward()                                   # backpropagate
self.optimizer.step()                             # Adam weight update
```

- `gather(dim=1, index=...)` picks, from each row of Q-values, the entry for the action that was actually taken in that transition.
- The `(1 - terminations)` factor is the $(1 - \gamma)$ term from the TD target: terminal transitions contribute only their immediate reward.
- Everything inside `torch.no_grad()` is treated as a constant — gradients flow only through `current_q`.

8.7 Hyperparameter Configuration

Each named block in `hyperparameters.yml` fully specifies a run; the Flappy Bird set looks like this:

```
flappybird1:
  env_id: FlappyBird-v0
  replay_memory_size: 100000
  mini_batch_size: 32
  epsilon_init: 1          # 1 = 100% random actions
  epsilon_decay: 0.99995   # epsilon decay rate
  epsilon_min: 0.05        # minimum epsilon value
  network_sync_rate: 10    # steps before syncing policy => target network
  learning_rate_a: 0.0001
  discount_factor_g: 0.99
  stop_on_reward: 100000
  fc1_nodes: 512
  dueling_hidden_dim: 256
  env_make_params:
    use_lidar: False
  enable_double_dqn: True
  enable_dueling_dqn: True
  device: auto
  seed: 42
```

Training and evaluation are launched from the command line:

```
python agent.py flappybird1 --train    # train (runs until stopped)
python agent.py flappybird1           # watch the best saved policy play
```

9. Discussion — Expected Training Behaviour

Because I have not yet run the full-length training this environment demands, this section describes what the training process does over time and what I expect to observe when it is run properly, based on the algorithm's mechanics and the hyperparameters chosen.

Early phase ($\epsilon \approx 1$). The agent plays essentially at random. In Flappy Bird random flapping dies almost immediately, so episode rewards should sit near the minimum (around -8 with this environment's death penalty and per-frame survival bonus) and the replay memory fills mainly with short, failed episodes. This phase is not wasted: the network is learning the easiest lesson in the game — which states immediately precede death — and that negative value information is the foundation everything else bootstraps from.

Middle phase (ϵ decaying). As epsilon decays, an increasing fraction of actions comes from the network's own Q-estimates, so the agent starts surviving longer, which produces better data, which improves the estimates further — a slow positive feedback loop. On the progress plot I expect the 100-episode rolling mean reward to climb from its floor, noisily: exploration and the randomised pipe gaps guarantee high variance, and the first time the agent passes a pipe ($+1.0$) will appear as isolated spikes before it becomes routine. The right-hand panel of the plot should show epsilon as a smooth exponential slide from 1.0 toward the 0.05 floor over roughly 60,000 optimisation steps.

Late phase ($\epsilon = \epsilon_{\min}$). With exploration at 5%, performance is dominated by the learned policy. The mean reward should continue to rise more steadily, and because the checkpoint is saved on

every new best episode, the model file on disk always captures the best behaviour found so far even if later training regresses. Training never stops on its own — the loop is infinite by design — so the run ends when I judge the reward curve to have plateaued.

The two environments should behave very differently. CartPole is nearly a sanity check: dense rewards (+1 every step), a 4-dimensional state, and simple dynamics mean the mean reward should climb within a few thousand episodes. Flappy Bird is the real test: sparse pipe rewards, a 12-dimensional state, and unforgiving collision dynamics mean tens of thousands of episodes before reliable pipe-passing emerges — and with this implementation's one-optimisation-per-episode design, learning per episode is slower than the per-step updates of the original DQN paper.

What failure would look like. If the mean reward is still flat after epsilon has substantially decayed, that is a diagnostic signal rather than just bad luck: it would point to the learning rate (too high → divergence, too low → glacial progress), an epsilon decay mismatched to the task, or too infrequent learning. The expected shape — flat, then accelerating, then steadily climbing — is the baseline against which I will judge the eventual real run.

10. Conclusion & Reflection

In this project I studied the Deep Q-Learning algorithm from the inside: starting from a working implementation copied from a tutorial, I worked backwards through every component until the code stopped being magic. The epsilon values that originally confused me turned out to be a simple, elegant idea — a probability of acting randomly, annealed from 1.0 to 0.05 on a schedule — and the seemingly redundant second network turned out to be the key stabilising trick that makes bootstrapped deep RL work at all.

The contrast with my previous ID3 project was instructive. ID3 is supervised, deterministic, and fully interpretable: the same data always yields the same tree, and every prediction can be read as a rule. DQN is the opposite on every axis: it generates its own data by acting, its training is stochastic and path-dependent, and its policy is buried in continuous weights. What it buys with that opacity is power — it learns behaviours nobody labelled, in state spaces no table could hold.

The honest gap in this project is empirical: I have understood the machine but not yet run it to completion. The clear next step is the long Flappy Bird training run, validated against the expectations set out in Section 9. Beyond that, the implementation invites experiments: swapping MSE for the Huber loss, moving optimisation from per-episode to per-step, and adding prioritised experience replay. Overall, this project moved me from copying reinforcement learning code to understanding why each of its parts exists — which was the real goal.

11. Appendix A — Glossary

- **Agent** — The learner and decision-maker — here, the neural network plus its action-selection rule.
- **Environment** — The world the agent acts in (the Flappy Bird game); it returns observations and rewards.
- **State / observation** — The vector of numbers describing the current situation (12 features in Flappy Bird).
- **Action** — A choice available to the agent; Flappy Bird has two: do nothing (0) or flap (1).

- **Reward (r)** — The scalar feedback after each action; the only learning signal in RL.
- **Episode** — One complete game, from reset to a terminal state (crash).
- **Return (G)** — The total discounted reward from a time step onward — the quantity the agent maximises.
- **Discount factor (γ)** — A number in $(0, 1)$ that shrinks future rewards; $\gamma = 0.99$ means rewards 100 steps away are worth ~37% of immediate ones.
- **Policy (π)** — The agent's rule for choosing actions given a state.
- **Q-value $Q(s, a)$** — The expected return of taking action a in state s , then acting well afterwards.
- **Epsilon (ϵ)** — The probability of taking a random action instead of the best-looking one.
- **Epsilon-greedy** — The exploration strategy: random action with probability ϵ , best action otherwise.
- **Epsilon decay** — The schedule that multiplies ϵ by a constant (0.99995) after each learning step, until a floor (0.05) is reached.
- **Exploration vs exploitation** — The core RL dilemma: try new things to discover better strategies, or use what you already know to score well.
- **Transition** — One stored experience: (state, action, next state, reward, done flag).
- **Replay memory / experience replay** — A large buffer of past transitions, sampled randomly for training to break temporal correlation and reuse data.
- **Mini-batch** — The set of transitions (32 here) sampled from replay memory for one gradient update.
- **Policy network** — The network being trained; it selects actions and produces the predictions in the loss.
- **Target network** — A frozen copy of the policy network used to compute stable learning targets; refreshed every few steps.
- **TD target (y)** — The regression target for a transition: reward plus discounted value of the next state.
- **Temporal-difference (TD) error** — The gap between the network's prediction and the TD target — what the loss minimises.
- **Bootstrapping** — Building learning targets partly from the model's own predictions of the future.
- **Overestimation bias** — The systematic inflation of Q-values caused by maximising over noisy estimates; addressed by Double DQN.
- **Advantage $A(s, a)$** — How much better an action is than the average action in that state; used by the dueling architecture.
- **Off-policy** — Learning about one policy (the greedy one) from data generated by another (the exploratory one).
- **Checkpoint** — The model weights saved to disk (runs/<set>.pt) whenever a new best episode reward is reached.

12. Appendix B — References

Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.

- Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., & Silver, D. (2018). Rainbow: Combining improvements in deep reinforcement learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1).
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *3rd International Conference on Learning Representations (ICLR)*. arXiv:1412.6980.
- Lin, L.-J. (1992). Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4), 293–321.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). Playing Atari with deep reinforcement learning. arXiv:1312.5602.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., et al. (2024). Gymnasium: A standard interface for reinforcement learning environments. arXiv:2407.17032.
- van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1). arXiv:1509.06461.
- Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., & de Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. arXiv:1511.06581.
- Watkins, C. J. C. H. (1989). *Learning from Delayed Rewards* (PhD thesis). University of Cambridge.
- Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4), 279–292.
- markub3327. *flappy-bird-gymnasium* [Computer software].
<https://github.com/markub3327/flappy-bird-gymnasium>